# The Order Processor Nobody Wants to Maintain

Clean Code refactoring of main.py — changes and decisions

```
- red: removed lines
+ green: added lines
```

OVERVIEW

# What we changed

1. Removed code
2. Naming
3. Magic numbers
4. Class extraction
5. Functions extraction
6. Global state
7. Money precision
8. Further cleanup: dead code
9. Further cleanup: naming
10. Discount split
11. Default values
12. handleCommand

Output stays identical after every step

1 · REMOVED CODE

# Remove code that gives no value

- Unused variables: mode, cache, future_currencies
- Unused parameter unused in doIt()
- Everything was an interface without value — Thing no longer extends it

```
@@ class Everything @@
-class Thing(Everything):
+class Thing:
@@ globals @@
-mode = "production"
-cache = {}
-future_currencies = ["EUR", "USD", "BTC", "POINTS"]
@@ def doIt @@
-def doIt(s, action, verbose, send, save, unused):
+def doIt(s, action, verbose, send, save):
-        cache[e] = x
@@ __main__ @@
-    doIt("Bob|bob@example.test|REGULAR|BAG:2|NONE", 1, True, True, True, 999)
+    doIt("Bob|bob@example.test|REGULAR|BAG:2|NONE", 1, True, True, True)
```

WHY

Obvious choice.

```
commit: Remove unused code
```

# Names that explain purpose

- Thing → Order

- doIt() → order() and showReport()

- a → orderInformation, z → price, x → orderDescription

```
@@ class @@
-class Thing:
+class Order:
@@ def doIt @@
-def doIt(s, action, verbose, send, save):
+def order(orderInformationString, verbose, send, save):
-        a = s.split("|")
-        z = 0
+        orderInformation = orderInformationString.split("|")
+        price = 0
-        if z < 100:
-            z += 12
-        z *= 1.23
-        x = f"ORDER|{n}|{detail}|{z:.2f}"
+        if price < 100:
+            price += 12
+        price *= 1.23
+        orderDescription = f"ORDER|{n}|{detail}|{price:.2f}"
@@ new @@
+def showReport():
+    print(f"REPORT|{counter}|{total:.2f}")
```

**WHY**
Obvious choice.

`commit: Improve naming`

# Named constants instead of numbers

- Prices → PRODUCT_PRICE
- 100, 12, 23% → FREE_DELIVERY_THRESHOLD, DELIVERY_COST, TAXES_PERCENTAGE
- 0.9 / 0.95 → DISCOUNTS

**WHY**

A name says what the number means for the business, and the value changes in one place.

`commit: Replace magic numbers with constants`

```
@@ def order @@
-            if c[0] == "BOOK":
-                p = 40
-            elif c[0] == "PEN":
-                p = 5
-            elif c[0] == "BAG":
-                p = 80
-            else:
-                p = 0
-            price += p * int(c[1])
+            price += PRODUCT_PRICE.get(c[0], 0) * int(c[1])
-        if t == "VIP":
-            price *= 0.9
-        elif t == "STUDENT":
-            price *= 0.95
-        if orderInformation[4] == "SAVE10":
-            price *= 0.9
-        if price < 100:
-            price += 12
-        price *= 1.23
+        price *= DISCOUNTS.get(MemberType[t], 1.0)
+        price *= DISCOUNTS.get(orderInformation[4], 1.0)
+        if price < FREE_DELIVERY_THRESHOLD:
+            price += DELIVERY_COST
+        price *= 1 + TAXES_PERCENTAGE
```

# Enums instead of codes

• Action codes 1, 2 → Action(Enum): ORDER, REPORT

• Member type strings → MemberType(Enum): REGULAR, VIP, STUDENT

**WHY**

Enums list every allowed value in one place; comparisons read as business terms, not codes.

`commit: Replace magic numbers with constants`

```
@@ def executeAction @@
-    if command == 1:
+    if command == Action.ORDER:
-    if command == 2:
+    if command == Action.REPORT:
@@ new @@
+class Action(Enum):
+    ORDER = 1
+    REPORT = 2
+
+class MemberType(Enum):
+    REGULAR = 1
+    VIP = 2
+    STUDENT = 3
+
+DISCOUNTS = {
+    "SAVE10": 0.9,
+    MemberType.REGULAR: 1.0,
+    MemberType.VIP: 0.9,
+    MemberType.STUDENT: 0.95,
+}
```

# Classes with one responsibility

- ProductsHolder: products, total price, description
- OrderDetails: parsed order data (name, email, ...)
- OrderProcessor: processes orders and reports

**WHY**

doIt() parsed, priced, counted, saved and printed at once (SRP); now each class has one reason to change.

`commit: Extract classes`

```
@@ class Order @@
+    class ProductsHolder:
+        def getTotalPrice(self):
+            return sum([PRODUCT_PRICE.get(product, 0) * quantity ...])
+    class OrderDetails:
+        def __extractDetails(self, orderStringRepresentation):
+            orderInformation = orderStringRepresentation.split("|")
+            self.name = orderInformation[0]
+            self.email = orderInformation[1]
+            self.memberType = MemberType[orderInformation[2]]
@@ def order @@
-def order(orderInformationString, verbose, send, save):
-    global counter, total
-        n, e, t = orderInformation[0], orderInformation[1], ...
-        for q in products:
-            c = q.split(":")
+class OrderProcessor:
+    def _order(self, orderInformationString, verbose, send, save):
+        obj = Order(orderInformationString)
+        details = obj.details
+        price = details.products.getTotalPrice()
```

# One small function per rule

- getProductPrice()
- applyDiscount()
- applyDeliveryCost()
- applyTaxes()

**WHY**

Each business rule gets a name and can be read and tested alone; order() reads like the README rules.

`commit: Extract functions`

```
@@ class OrderProcessor @@
-         price *= DISCOUNTS.get(details.memberType, 1.0)
-         price *= DISCOUNTS.get(details.discountCode, 1.0)
+         price = applyDiscount(price, details.memberType)
+         price = applyDiscount(price, details.discountCode)
-         if price < FREE_DELIVERY_THRESHOLD:
-             price += DELIVERY_COST
-         price *= 1 + TAXES_PERCENTAGE
+         price = applyDeliveryCost(price)
+         price = applyTaxes(price)
@@ new @@
+def applyDiscount(price, discountReason):
+    return price * DISCOUNTS.get(discountReason, 1.0)
+
+def applyDeliveryCost(price):
+    if price < FREE_DELIVERY_THRESHOLD:
+        price += DELIVERY_COST
+    return price
+
+def applyTaxes(price):
+    return price * (1 + TAXES_PERCENTAGE)
```

# No more global variables

- counter and total are now fields of OrderProcessor

- Each processor has its own state

```
@@ globals @@
-counter = 0
-total = 0
-
-def order(orderInformationString, verbose, send, save):
-    global counter, total
@@ class OrderProcessor @@
+class OrderProcessor:
+    def __init__(self):
+        self.counter = 0
+        self.total = 0
```

**WHY**

Global counters leak between runs and tests; a processor that owns its state can be tested in isolation.

`commit: Extract classes`

# Decimal instead of float

• Prices, discounts, delivery cost and taxes use Decimal

```
@@ imports @@
+from decimal import Decimal
@@ PRODUCT_PRICE @@
-        "BOOK": 40,
+        "BOOK": Decimal(40),
@@ DISCOUNTS @@
-    MemberType.VIP: 0.9,
-    MemberType.STUDENT: 0.95,
+    MemberType.VIP: Decimal(0.9),
+    MemberType.STUDENT: Decimal(0.95),
@@ constants @@
-FREE_DELIVERY_THRESHOLD = 100
-DELIVERY_COST = 12
-TAXES_PERCENTAGE = 0.23
+FREE_DELIVERY_THRESHOLD = Decimal(100)
+DELIVERY_COST = Decimal(12)
+TAXES_PERCENTAGE = Decimal(0.23)
```

**WHY**

Binary float cannot store 0.1 exactly; money needs exact decimal arithmetic.

`commit: Use Decimal for money`

# Delete dead code

- Commented-out class Everything — commented code is still dead code

- refund() — neither implemented nor used

```
@@ top of file @@
-# from abc import ABC, abstractmethod
-# class Everything(ABC):
-#     @abstractmethod
-#     def send(self, x):
-#         pass
-#     @abstractmethod
-#     def save(self, x):
-#         pass
-#     @abstractmethod
-#     def refund(self, x):
-#         pass
@@ class Order @@
-    def refund(self, x):
-        raise NotImplementedError("Not supported")
```

**WHY**

Obvious choice.

`commit: Remove leftover unused code`

# Rename remaining vague names

- obj → order
- x → message, orderDescription
- detail → productsDescription

```
@@ class Order @@
-    def send(self, x):
-        print("EMAIL " + x)
+    def send(self, message):
+        print("EMAIL " + message)
-    def save(self, x):
+    def save(self, orderDescription):
@@ class OrderProcessor @@
-        obj = Order(orderInformationString)
-        details = obj.details
+        order = Order(orderInformationString)
+        details = order.details
-        detail = str(details.products)
+        productsDescription = str(details.products)
-                obj.save(orderDescription)
+                order.save(orderDescription)
```

**WHY**

Obvious choice.

`commit: Improve remaining names`

# Coupons are not member types

• DISCOUNTS → MEMBER_DISCOUNTS and COUPON_DISCOUNTS

• applyDiscount() → applyMemberDiscount() and applyCouponDiscount()

```
@@ DISCOUNTS @@
-DISCOUNTS = {
-    "SAVE10": Decimal(0.9),
+MEMBER_DISCOUNTS = {
     MemberType.REGULAR: Decimal(1.0),
     MemberType.VIP: Decimal(0.9),
     MemberType.STUDENT: Decimal(0.95),
 }
+
+COUPON_DISCOUNTS = {
+    "SAVE10": Decimal(0.9),
+}
@@ functions @@
-def applyDiscount(price, discountReason):
+def applyMemberDiscount(price, memberType):
+def applyCouponDiscount(price, couponCode):
@@ class OrderProcessor @@
-        price = applyDiscount(price, details.memberType)
-        price = applyDiscount(price, details.discountCode)
+        price = applyMemberDiscount(price, details.memberType)
+        price = applyCouponDiscount(price, details.discountCode)
```

**WHY**

Member types and coupons change for different reasons; one shared dict could also mix up their keys.

`commit: Split coupons from member discounts`

# No more True, True, True

- verbose, send and save default to True
- Callers pass only what differs

```
@@ __main__ @@
-    processor.executeAction(
-        Action.ORDER, "Bob|bob@example.test|REGULAR|BAG:2|NONE", True, True, True
-    )
+    processor.executeAction(Action.ORDER, "Bob|bob@example.test|REGULAR|BAG:2|NONE")
```

**WHY**

Callers with the standard behavior no longer repeat True, True, True; only differences stay visible.

`commit: Use default values for flags`

# Command handling outside order logic

- executeAction() moved out of OrderProcessor as handleCommand()
- Text commands ORDER <order line> and REPORT replace Action(Enum)
- Streaming input can still call operations generically

```
@@ class OrderProcessor @@
-    def _order(self, orderInformationString, verbose, send, save):
+    def order(self, orderInformationString, verbose=True, send=True, save=True):
-    def _showReport(self):
+    def report(self):
-    def executeAction(self, command, info="", verbose=True, send=True, save=True):
-        if command == Action.ORDER:
-            return self._order(info, verbose, send, save)
@@ new @@
+def handleCommand(processor, commandLine):
+    command, _, payload = commandLine.partition(" ")
+    if command == "ORDER":
+        return processor.order(payload)
+    if command == "REPORT":
+        return processor.report()
+    return "UNKNOWN"
@@ __main__ @@
-    processor.executeAction(Action.REPORT)
+    handleCommand(processor, "REPORT")
```

**WHY**

OrderProcessor should not know the input format; commands can come from any source.

`commit: Move command handling out of OrderProcessor`

OVERVIEW

# What we changed next

13. Decimal from strings
14. Price calculation
15. Dependency injection
16. Constants first
17. Dataclasses
18. Error handling
19. Count after saving
20. Return values
21. snake_case

Demo output stays identical after every step

# Decimal without float errors

- Decimal(0.9) keeps the float error: 0.90000000000000222…
- Decimal("0.9") is exactly 0.9
- Same for 0.95, 0.23 and 1.0

```
@@ MEMBER_DISCOUNTS @@
-    MemberType.REGULAR: Decimal(1.0),
-    MemberType.VIP: Decimal(0.9),
-    MemberType.STUDENT: Decimal(0.95),
+    MemberType.REGULAR: Decimal("1.0"),
+    MemberType.VIP: Decimal("0.9"),
+    MemberType.STUDENT: Decimal("0.95"),
@@ COUPON_DISCOUNTS @@
-    "SAVE10": Decimal(0.9),
+    "SAVE10": Decimal("0.9"),
@@ def applyMemberDiscount @@
-    return price * MEMBER_DISCOUNTS.get(memberType, Decimal(1.0))
+    return price * MEMBER_DISCOUNTS.get(memberType, Decimal("1.0"))
@@ constants @@
-TAXES_PERCENTAGE = Decimal(0.23)
+TAXES_PERCENTAGE = Decimal("0.23")
```

**WHY**

Decimal(0.9) copies the float error, so step 7 did not fully fix money precision.

`commit: Create Decimal values from strings`

# Pricing rules in one function

- calculatePrice() extracted from order()
- Pure business logic: no files, no console
- order() only puts the steps together

```
@@ class OrderProcessor @@
-         price = details.products.getTotalPrice()
+         price = calculatePrice(details)
          productsDescription = str(details.products)
-
-         price = applyMemberDiscount(price, details.memberType)
-         price = applyCouponDiscount(price, details.discountCode)
-
-         price = applyDeliveryCost(price)
-         price = applyTaxes(price)
@@ new @@
+def calculatePrice(details):
+    price = details.products.getTotalPrice()
+    price = applyMemberDiscount(price, details.memberType)
+    price = applyCouponDiscount(price, details.discountCode)
+    price = applyDeliveryCost(price)
+    return applyTaxes(price)
```

**WHY**

Pricing is the core business rule; as a pure function it is tested without files or console.

`commit: Extract price calculation`

# Side effects come from outside

- send() → ConsoleNotifier, save() → FileOrderRepository
- OrderProcessor receives them in the constructor (DIP)
- Tests can pass fakes — send and save flags removed

```
@@ class Order @@
-    def send(self, message):
-        print("EMAIL " + message)
-    def save(self, orderDescription):
-        with Path("orders.txt").open("a", encoding="utf-8") as f:
@@ new @@
+class FileOrderRepository:
+    def __init__(self, path):
+        self.path = path
+    def save(self, orderDescription):
+        with self.path.open("a", encoding="utf-8") as f:
+class ConsoleNotifier:
+    def send(self, email, message):
+        print("EMAIL " + email + "|" + message)
@@ class OrderProcessor @@
-    def __init__(self):
+    def __init__(self, repository, notifier):
-    def order(self, orderInformationString, verbose=True, send=True, save=True):
+    def order(self, orderInformationString, verbose=True):
-            if save:
-                order.save(orderDescription)
-            if send:
-                order.send(details.email + "|" + orderDescription)
+            self.repository.save(orderDescription)
+            self.notifier.send(details.email, orderDescription)
@@ __main__ @@
-    processor = OrderProcessor()
+    processor = OrderProcessor(FileOrderRepository(Path("orders.txt")), ConsoleNotifier())
```

**WHY**

Business logic should not depend on files and console (DIP); fakes make tests fast and isolated.

```
commit: Inject repository and
notifier into OrderProcessor
```

# Definitions before use

- MemberType, discounts, delivery and taxes moved to the top
- PRODUCT_PRICE is a module constant, not rebuilt on every call

```
@@ def getProductPrice @@
-def getProductPrice(product):
-    PRODUCT_PRICE = {
-        "BOOK": Decimal(40),
-        "PEN": Decimal(5),
-        "BAG": Decimal(80),
-    }
-
-    return PRODUCT_PRICE.get(product, 0)
+PRODUCT_PRICE = {
+    "BOOK": Decimal(40),
+    "PEN": Decimal(5),
+    "BAG": Decimal(80),
+}
+
+def getProductPrice(product):
+    return PRODUCT_PRICE.get(product, 0)
@@ moved to the top of the file @@
 class MemberType(Enum): ...
 MEMBER_DISCOUNTS = { ... }
 COUPON_DISCOUNTS = { ... }
 FREE_DELIVERY_THRESHOLD = Decimal(100)
 DELIVERY_COST = Decimal(12)
 TAXES_PERCENTAGE = Decimal("0.23")
```

**WHY**

Readers meet definitions before uses, and the dataclass needs MemberType defined above it.

`commit: Move constants to the top`

# Order is plain data

- Nested ProductsHolder and OrderDetails → @dataclass Order and OrderLine
- parseOrder(): one place that knows the text format
- No more [product, quantity] lists

**WHY**

Order was only a wrapper around details; a dataclass shows the data and parsing lives in one place.

`commit: Replace nested classes with dataclasses`

```
@@ class Order @@
-    def __init__(self, orderStringRepresentation):
-        self.details = self.OrderDetails(orderStringRepresentation)
-    class ProductsHolder: ...
-    class OrderDetails: ...
+@dataclass
+class OrderLine:
+    product: str
+    quantity: int
+
+@dataclass
+class Order:
+    name: str
+    email: str
+    memberType: MemberType
+    lines: list[OrderLine]
+    couponCode: str
@@ new @@
+def parseOrder(orderStringRepresentation):
+    orderInformation = orderStringRepresentation.split("|")
+    return Order(name=orderInformation[0], ...)
@@ class OrderProcessor @@
-        order = Order(orderInformationString)
-        details = order.details
+        order = parseOrder(orderInformationString)
```

# Catch only what we expect

- except Exception hid every error behind ERROR
- Only storage errors (OSError) return ERROR now
- Parse errors are no longer hidden

```
@@ class OrderProcessor @@
          try:
              self.repository.save(orderDescription)
              self.notifier.send(order.email, orderDescription)
              if verbose:
                  print(orderDescription)
              return orderDescription
-         except Exception:
+         except OSError:
              return "ERROR"
```

**WHY**

Catching everything turned bugs into ERROR; callers could not tell bad input from a storage failure.

`commit: Catch only storage errors`

# No counted order without saving

- counter and total were updated before saving
- A failed save no longer counts the order in the report
- Only saved orders send a notification

**WHY**

The report counted orders that were never stored; a retry could count the same order twice.

`commit: Count orders only after saving`

```
@@ class OrderProcessor @@
-         self.counter += 1
-         self.total += price
-
          try:
              self.repository.save(orderDescription)
-             self.notifier.send(order.email, orderDescription)
-             if verbose:
-                 print(orderDescription)
-             return orderDescription
          except OSError:
              return "ERROR"
+
+         self.counter += 1
+         self.total += price
+         self.notifier.send(order.email, orderDescription)
+         if verbose:
+             print(orderDescription)
+         return orderDescription
```

# Return results, print in one place

- order() and report() both return strings
- __main__ prints the results
- verbose flag removed

```
@@ class OrderProcessor @@
-    def order(self, orderInformationString, verbose=True):
+    def order(self, orderInformationString):
-        if verbose:
-            print(orderDescription)
         return orderDescription
@@ def report @@
-        print(f"REPORT|{self.counter}|{self.total:.2f}")
+        return f"REPORT|{self.counter}|{self.total:.2f}"
@@ __main__ @@
-    handleCommand(processor, "ORDER Alice|...|VIP|BOOK:2,PEN:3|SAVE10")
-    handleCommand(processor, "ORDER Bob|...|REGULAR|BAG:2|NONE")
-    handleCommand(processor, "REPORT")
+    print(handleCommand(processor, "ORDER Alice|...|VIP|BOOK:2,PEN:3|SAVE10"))
+    print(handleCommand(processor, "ORDER Bob|...|REGULAR|BAG:2|NONE"))
+    print(handleCommand(processor, "REPORT"))
```

**WHY**

One operation printed and the other returned; now both return and output happens in one place.

`commit: Return results instead of printing them`

21 · SNAKE_CASE

# PEP 8 naming

- camelCase was left over from the Java version
- orderStringRepresentation, orderInformationString → order_text
- Classes stay in CapWords: Order, OrderLine

**WHY**

Obvious choice.

`commit: Use snake_case names`

```
@@ class Order @@
-    memberType: MemberType
-    couponCode: str
+    member_type: MemberType
+    coupon_code: str
-    def getSubtotal(self):
+    def get_subtotal(self):
@@ class OrderProcessor @@
-    def order(self, orderInformationString):
-        order = parseOrder(orderInformationString)
-        price = calculatePrice(order)
+    def order(self, order_text):
+        order = parse_order(order_text)
+        price = calculate_price(order)
@@ functions @@
-def applyMemberDiscount(price, memberType):
-def applyCouponDiscount(price, couponCode):
-def applyDeliveryCost(price):
-def applyTaxes(price):
+def apply_member_discount(price, member_type):
+def apply_coupon_discount(price, coupon_code):
+def apply_delivery_cost(price):
+def apply_taxes(price):
```

RESULT

# Same behavior, cleaner code

- Demo output identical after every commit
- No dead code, no magic numbers
- Each class and function has one job
- No global state
- Side effects injected, errors not hidden

```
$ python3 main.py
EMAIL alice@example.test|ORDER|Alice|BOOKx2;PENx3;|109.41
ORDER|Alice|BOOKx2;PENx3;|109.41
EMAIL bob@example.test|ORDER|Bob|BAGx2;|196.80
ORDER|Bob|BAGx2;|196.80
REPORT|2|306.21
```